DressMe BO7 — Virtual Try-On Research Report

Oct 8, 2026 · @klod

Summary

BO7 selected **FASHN VTON v1.5 in float16 with 15 sampling steps**, at about 37 s per try-on on a Tesla T4, down from 448 s at the start (12x faster). Two pretrained try-on models, FASHN v1.5 and CatVTON-mix, were compared on 12 controlled cases; neither was a clear quality winner, and FASHN was chosen because it needs no mask for the user photo.

No model was trained or fine-tuned, so the project owns no trained `.pt` weights: the deliverables are two reproducible inference bundles that download the pretrained weights, plus the research evidence (section 8). On 8 real BO6 wardrobe garments, 7 results were usable. The contribution is a data audit, a controlled model comparison, a 12x runtime optimisation, a real-wardrobe test and an evidence-based model choice; the work log lists the 12 problems solved along the way.

Objective and scope

BO7 virtual try-on takes a photo of the user and a garment from the BO6 digital wardrobe, and generates an image of the user wearing that garment. This phase answered one question: which pretrained try-on model, at which settings, should DressMe use on the available hardware?

The study was pretrained-model-first: no try-on model was trained from scratch or fine-tuned. Hybrid Avatar, also part of BO7, is a separate milestone and is not covered here.

Data

Two datasets were used: the BO6 catalogue as the source of wardrobe garments, and VITON-HD for controlled person-and-garment test cases.

BO6 catalogue (Kaggle fashion-product-images-dataset). The dataset ships its image tree twice; the audit used the copy that holds the selected `styles.csv`.

Check	Result
Images scanned	44,441

Check	Result
Corrupt images	0
Duplicate image IDs	0
Metadata rows	44,424 (22 malformed rows quarantined)
Rows in exact-duplicate content groups	1,431 (779 redundant copies in 652 groups)
Try-on candidate garments	15,354: 12,515 tops, 2,384 bottoms, 455 one-pieces

One candidate row is a belt mislabelled `Tshirts`; the BO6-to-BO7 connection should keep only rows whose `masterCategory` is `Apparel`.

VITON-HD (Kaggle mirror by Marquis03, CC BY-NC 4.0, academic use). The mirror has no binary agnostic masks, so masks were derived by comparing each person photo with its `agnostic-v3.2` image: grey fill 128 ± 14 near pixels that changed by more than 24, then a median filter and a 5-pixel dilation. All 4,000 sampled pairs (2,000 from the train split, 2,000 from test) passed, with 0 rejected. Visual checks of 14 overlays, including the 8 most extreme mask sizes, showed the masks covering torso, arms and neck while preserving face and lower body; long hair over the shoulders falls inside the mask.

The pairs were split by content groups into 256 training, 24 validation and 6 test cases; 8 pairs that appeared in both official splits were quarantined.

Method

Two pretrained models were run on the same 12 frozen cases, each in its own isolated Python environment, with seed 42 and only one of the two T4 GPUs.

Model	Source revision	Needs a mask	Bundled licence
FASHN VTON v1.5	commit `7c0f10a`	No	Apache 2.0 (code)
CatVTON-mix (SD 1.5 inpainting)	commit `7818397`, checkpoint `mix-48k-1024`	Yes, aligned binary mask	CC BY-NC-SA 4.0

- **Cases:** 6 paired cases, where the person already wears the target garment, and 6 changed-garment cases, where a different garment replaces it. Only the changed-garment cases represent the product; paired cases favour FASHN, which sees the answer because it uses no mask.

- **Recorded per run:** time per image, peak GPU memory (PyTorch only), output size, settings, source and checkpoint revisions.
- **Quality:** visual review of the outputs on a blinded contact sheet, judged on garment fidelity, preservation of the person and other clothes, and artifacts.
- **Environment:** Kaggle, Python 3.13, PyTorch 2.11 with CUDA 12.8, 2 Tesla T4 (15 GB each).

Work log: problems found and fixed

Twelve problems came up between the first review and the final export; each was traced to a cause and fixed before moving on. They are listed in the order met.

#	Stage	Problem	Cause	Fix	Effect
1	Notebook review	Plan built around a LoRA run unlikely to show a gain (lr 1e-5, 300 updates, data the model had already seen)	Design, before any run	Learning rate raised to 1e-4; LoRA kept behind gates	LoRA later dropped (Decision)
2	Setup	Creating the isolated environments crashed	Kaggle's Python cannot bootstrap pip into a new venv	Venv built without pip, reusing the kernel's pip and Torch	Both environments built
3	Setup	First full run had no GPU	Session started without the accelerator	Rerun on 2x T4	GPU stages possible
4	BO6 audit	0 candidate garments	Dataset ships its image tree twice, so every ID looked duplicated	Scan only the copy holding `styles.csv`; collapse the mirror	15,354 candidates; scan time halved
5	VITON-HD	All 4,000 pairs rejected	Mirror has no binary agnostic masks	Masks derived from `agnostic-v3.2`, then reviewed by eye	4,000 valid pairs
6	FASHN	Pose models ran on CPU	ONNX Runtime expected CUDA 13; Torch uses CUDA 12.8	Isolated ONNX Runtime 1.26.0 (CUDA 12)	Pose on GPU; total time unchanged
7	FASHN	448 s per image	bfloat16 is only emulated on the T4	Override to float16, checked against the bfloat16 image	65 s, 6.8x faster
8	CatVTON	One clothed result blocked as NSFW	Built-in safety checker false positive	Checker turned off for the benchmark	All cases produce images

#	Stage	Problem	Cause	Fix	Effect
9	CatVTON	Logo garbled, sleeve missing, hems cropped	Run at 384x512, far below its 1024-pixel training size	Rerun at 768x1024	Quality near FASHN, time ~73 s
10	Speed	Both models ~72 s per image	30 sampling steps	Speed sweep; FASHN set to 15 steps	36.6 s, no visible loss
11	BO6 test	Orange T-shirt got long sleeves and white trousers	Flat-lay photo run as "worn by a model"	Photo type set per garment	7 of 8 garments usable
12	Export	FASHN bundle would fail to restore	Checksums included Hugging Face cache files that change on every download	Checksums limited to the 3 weight files	Bundle restorable

Runtime engineering

Switching FASHN from bfloat16 to float16 made it 6.8x faster with no visible change, and was the single largest gain of the project.

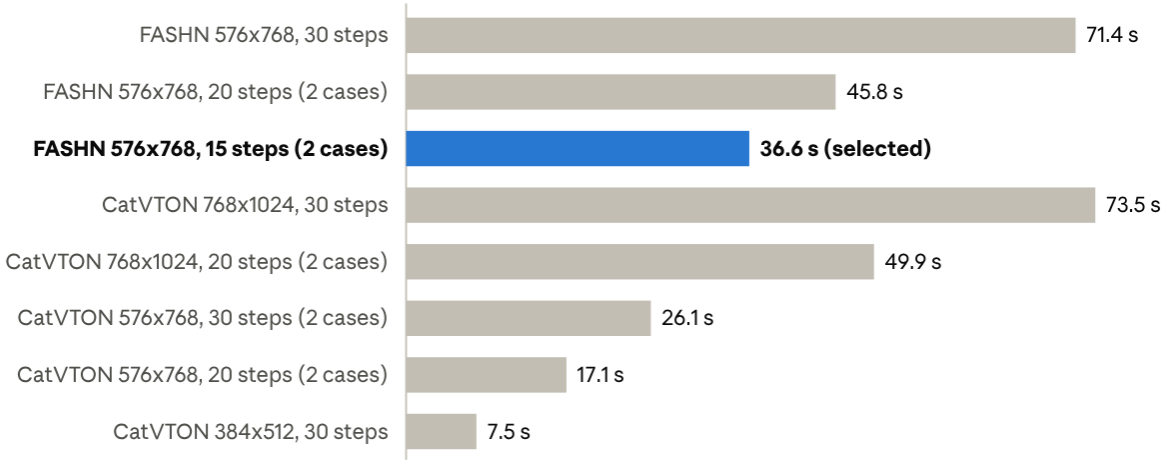
1. **Pose models on GPU.** FASHN's ONNX pose models first fell back to CPU because the installed ONNX Runtime expected CUDA 13 libraries. An isolated environment with ONNX Runtime 1.26.0 (CUDA 12) put both pose sessions on the GPU. Speed did not change: diffusion sampling takes 99.6% of FASHN's time.
2. **Precision.** FASHN's pipeline selects bfloat16 automatically (`pipeline.py`, lines 72–75), which the T4 only emulates. On the official sample, float16 cut time from 447.9 s to 65.5 s and peak memory from 10.90 GiB to 2.96 GiB; the two images differ by 0.28 on a 0–255 scale on average.
3. **CatVTON safety filter.** The built-in NSFW checker blocked one result showing a plain white T-shirt. It was turned off for the benchmark; the block is kept in the logs.

Results

All 36 benchmark runs succeeded (12 cases for each of three configurations), and at matched resolution the two models cost about the same time.

FASHN at 15 steps takes 36.6 s per image, half its 30-step time

Median seconds per image, one Tesla T4, FASHN in float16. The project started at 447.9 s (FASHN, bfloat16).



bo7_research_evidence runs/*.jsonl · 12-case benchmark medians; rows marked 2 cases come from the speed sweep

Peak GPU memory stayed under 3.6 GiB in every float16 run: FASHN 2.96 GiB, CatVTON 2.80 GiB at 384x512 and 3.57 GiB at 768x1024. FASHN outputs were 576x768 on these inputs. No visible quality loss was seen between 30, 20 and 15 steps for either model on the 2 speed-sweep cases. A follow-up run of FASHN at 15 steps on all 6 changed-garment cases took a median 34.7 s and matched the 30-step result in 5 of 6; in the camisole case, already a failure at 30 steps, the buttons came out misaligned.

Quality. At 384x512, CatVTON was ten times faster but visibly worse: garbled logo, a missing sleeve, cropped hems and leftovers of the old garment. At 768x1024 most of these disappeared. On the 6 paired cases both models looked good. The 6 changed-garment cases are the ones that matter:

Changed-garment case	CatVTON 768x1024	FASHN	Better
Grey top to burgundy bodysuit	Both sleeves; neckline closer to garment	Clean; deeper V-neck	Even (2nd reviewer: CatVTON)
White tee to dark long-sleeve	Too light; cropped at the waist	Full length; too black	FASHN
Curly hair to white tee	Clean shirt; hair outline changed	Small gap at the hem	Even
Burgundy top to logo T-shirt	Logo fainter	Logo readable	FASHN

Changed-garment case	CatVTON 768x1024	FASHN	Better
Floral top to cream camisole	Straps and buttons right; small floral cuff left	Turned into a waistcoat; trousers repainted white	CatVTON
Blonde to floral top	Scoop neckline matches	Neckline changed to a V	CatVTON

The failure patterns differ: CatVTON struggles at the waistline, at the lower edge of its mask, while FASHN, having no mask, can repaint clothes it should leave alone.

Real wardrobe test (BO6 garments)

FASHN at 15 steps dressed one VITON-HD person in 8 random BO6 catalogue garments: all 8 succeeded at a median 35.9 s per image, and 7 looked usable once each garment's photo type was set correctly.

BO6 garment (ID)	Catalogue photo	Result
Orange T-shirt, top (9927)	Flat-lay	Good shape and cap sleeves; print blurred. Run as "worn" first, it got long sleeves and white trousers
Red ruffled top (41767)	Worn by a model	Good; ruffles kept
Purple printed T-shirt (34351)	Worn by a model	Good; large print kept, small text garbled
Yellow blouse (37913)	Worn by a model	Good
White printed long-sleeve (24730)	Worn by a model	Print kept; trousers replaced by the jeans from the catalogue photo
Track pants, bottom (43081)	Flat-lay	Good; logo kept
Light trousers, bottom (43917)	Worn by a model	Good
Red dress, one-piece (5007)	Flat-lay	Good

Two product lessons follow. BO6 must record whether each garment photo is flat-lay or worn, because the wrong setting broke the orange T-shirt. And when a catalogue photo shows a model, FASHN can copy that model's other clothes onto the user, as with the jeans; printed text on garments is also not reproduced legibly.

Decision

DressMe uses **FASHN v1.5, float16, 15 sampling steps**: about 37 s per image on a T4, no mask needed, quality comparable to CatVTON on the cases reviewed.

CatVTON-mix at 576x768 with 20 steps is the documented fast alternative, at 17.1 s per image. It was not chosen as the main model because every user photo would need an aligned mask: DressMe has no automatic masker yet, and the time to make a mask is not included in that 17.1 s. Its licence is also non-commercial.

LoRA fine-tuning was not pursued. CatVTON's largest gain came from fixing its resolution, and its remaining errors sit at the mask edge; a short LoRA on VITON-HD would not address either. The notebook's trainer exists but was never run.

Deliverables

There is no `.pt` model of our own: nothing was trained, so the try-on runs on the authors' pretrained weights, which the bundles download and verify.

File	What it holds
`fashn_inference_bundle.zip`	Selected model: inference code, model card, settings (float16, 15 steps), source commit, weight checksums, `restore_bundle.py`
`catvton_base_inference_bundle.zip`	Alternative: inference code, model card, exact checkpoint revisions, `restore_bundle.py`
`bo7_research_evidence.zip`	Inventories, manifests, 4,000 masks, run logs, contact sheets, speed sweep, `model_selection.json`
Executed Kaggle notebook	Every step with its outputs

The weights themselves (several GB) are not inside the bundles. The FASHN bundle was rebuilt so its checksums cover only the three weight files; the notebook's export also lists download-cache files, which change on every download and would stop a restore. Use the rebuilt version, or apply the one-line fix in Cell 05 (skip files under `.cache`) before exporting again.

The weights a restore fetches are FASHN's `model.safetensors` plus two ONNX pose models, and for CatVTON the SD inpainting UNet, the CatVTON attention weights and the `sd-vae-ft-mse` VAE.

How to reproduce

A try-on with the selected model can be rerun from the FASHN bundle alone, on any machine with an NVIDIA GPU and internet access.

1. Use Python 3.13 with PyTorch 2.11 for CUDA 12.8, as recorded in `kernel_environment.json`.
2. Unzip `fashn_inference_bundle.zip` and install its `requirements.txt` with `torch_constraints.txt`, so PyTorch is not replaced.
3. Run `python restore_bundle.py experiment_config.json`. It checks out FASHN at commit `7c0f10a`, downloads the weights and verifies their checksums.
4. Fill `input_template.csv` with a person photo, a garment photo, the category (`tops`, `bottoms` or `one-pieces`) and the photo type (`flat-lay` or `model`).
5. Run `python fashn_worker.py experiment_config.json input_template.csv my_run`. The config already sets float16 and 15 steps; the result lands in `reproduced_outputs/`.

The CatVTON bundle works the same way with `cat_worker.py`, but each row also needs an aligned binary mask (white = area to replace). To rerun the whole study, open the executed Kaggle notebook and run it top to bottom with the same datasets attached.

Limitations

The timings are measured; the quality comparison is a small visual judgement and should be read as such.

- Only 6 changed-garment cases; two reviewers disagreed slightly (2-2-2 against 3-2-1 for CatVTON), and both found no decisive winner.
- The structured review sheets were left blank, so there are no recorded scores.
- The 15-step setting was checked on 6 changed-garment cases and 8 BO6 garments, with one person; float16 against bfloat16 on 1 sample.
- No numeric quality metric: changed-garment cases have no reference image, and FID or KID mean nothing at 12 images.
- CatVTON used derived masks, not its own masker, and ran with its safety filter off.
- The pretrained models may have seen VITON-HD during their own training.
- Timings are for a Kaggle T4 and do not predict other hardware.

- VITON-HD and CatVTON are non-commercial; FASHN's code is Apache 2.0, but its weights' terms need checking before non-academic use.

Next steps

- [ ] Package the FASHN bundle as the BO7 inference worker, with job status, errors and retries.
- [ ] Connect it to BO6: garment ID, image, category and photo type (BO6 must store flat-lay or worn per garment); keep only `Apparel` rows.
- [ ] Store user photos and results privately, with deletion controls.
- [ ] Optional: fill the review sheets with two or three blind reviewers to back the quality claim.
- [ ] Hybrid Avatar: separate BO7 milestone, not started here.

Key terms

Term	Meaning in this report
Virtual try-on (VTON)	Generating an image of a person wearing a garment from a separate photo of that garment
Paired case	The person already wears the target garment, so a correct answer image exists
Changed-garment case	A different garment replaces the one worn; no correct answer image exists
Mask	Black-and-white image marking the area to replace (white) and to keep (black); CatVTON needs one, FASHN does not
Flat-lay / worn	Garment photographed on its own, or worn by a model in the catalogue photo
Sampling steps	Denoising iterations of the diffusion model; fewer steps means faster, with a risk of lost detail
bfloat16 / float16	16-bit number formats; the T4 runs float16 natively but only emulates bfloat16
LoRA	Light fine-tuning that adds small trainable matrices to a frozen model; planned but not run

Term	Meaning in this report
Inference bundle	Code, settings and checksums that download and rebuild a pretrained model exactly; contains no weights